Saga Pattern

Category: Data consistency / distributed transactions **One-liner:** Break a business transaction that spans several services into a sequence of local transactions, each with a compensating action that undoes it — achieving eventual consistency without **distributed locks** (locks held across several machines at once so no one else can change the data).

> This chapter is the book's home for the **Transactional Outbox** (Part 13D); Event-Driven and others link here.

Part 0 — Concepts you need first

What is a transaction?

A **transaction** is one logical operation of one or more data steps that **succeed or fail together**. A bank transfer — subtract £100 from A, add £100 to B — must be **one indivisible unit**, or £100 vanishes.

What is ACID?

The four guarantees of a traditional database transaction (§ Transactions and ACID):

Property	Meaning
Atomicity	Succeeds completely or fails completely; any failure leaves the database unchanged. *Atomic = indivisible.*
Consistency	Moves the database from one **valid state to another**, respecting all rules and constraints
Isolation	**Concurrent** transactions end in the same state as if run **one after another**; your half-finished work is invisible to others
Durability	Once **committed**, it **stays committed**, even if power fails a millisecond later

Together: a completed transaction **either fully occurred or didn't occur at all**.

What is a "rollback"?

Undoing a transaction's changes back to the pre-transaction state. In one database it's automatic and free — the database keeps a log and reverses it. **That's the capability you lose when a transaction spans services**; replacing it by hand is what Saga is about.

What is a database lock?

While a transaction modifies a row, the database **locks** it so nobody else can — that's how isolation works. **Locks are held until commit**: milliseconds for a short transaction, but seconds if it waits on a network call to another continent — during which nobody else can touch that row.

What is "database per service"?

A core **microservices** principle: **each service owns its database**, and no other service reads or writes it directly — Order Service *asks* Inventory Service rather than querying its tables. This gives independence (Inventory can change its **schema** freely), but it's precisely what rules out one database transaction spanning both.

What is eventual consistency?

The system **may temporarily be inconsistent** but **eventually converges**. Strong consistency: the moment the transfer completes, every observer sees both accounts updated. Eventual: briefly, an observer might see the debit but not yet the credit. Distributed systems accept this because strong consistency across a network needs locks and coordination that cost **availability** and speed (§ Consistency models, § CAP).

Part 1 — The problem

It works fine in a monolith

Checkout = adjust inventory, create order, take payment. In a **monolith** with one database that's one ACID transaction: if payment fails, inventory and order roll back automatically. **Atomicity for free.**

Why it breaks in microservices

Coordination. Inventory, Order and Payment run on different servers, each with its **own database**. Atomicity means undoing earlier steps when any fails — and **coordinating that rollback across independent services is genuinely hard**: no shared transaction log, no single database to tell "undo everything".

Network latency. Services may be geographically distributed, so a transaction can take a long time, and **holding locks that long is impractical**.

> A database transaction normally holds locks for milliseconds; a distributed one across three services might hold them for **seconds**, during which every other customer buying the same product is blocked. At real traffic, the system grinds to a halt.

Conclusion: ACID's model **doesn't translate to services that operate independently and communicate over networks.**

Part 2 — The solution

A saga is a sequence of local transactions. Each one:

1. **Updates its own database** (a normal, fully ACID local transaction)
2. **Publishes an event** (a message announcing what happened) that triggers the next local transaction

If a local transaction **fails because it violates a business rule** (e.g. "stock can't go below zero"), the saga runs **compensating transactions** that undo the preceding local transactions. Rather than enforcing strict consistency, Saga accepts **eventual consistency**.

The guarantee it actually gives

Either **all transactions complete**, or **compensating transactions roll back** the ones already executed — integrity through explicit compensation rather than automatic rollback. Ideal for **long-lived business transactions spanning multiple microservices** and globally distributed, **highly available** databases.

The cost: designing sagas is complex — intricate coordination and **extensive error handling**. You're hand-writing what the database used to do for you. (The course's image: ACID is the internal wiring of each building; a saga is the power line connecting buildings.)

Part 3 — The two types

Choreography — decentralized

Every local transaction **publishes domain events** (business facts, like `OrderCreated`) that trigger local transactions elsewhere. **No central coordination**: each service listens to others' events and decides whether to act (an orchestra with no conductor).

In practice: Order publishes `OrderCreated` → Payment charges and publishes `PaymentCompleted` → Inventory reserves stock and publishes `StockReserved`. Nobody is in charge; the flow emerges from who **subscribes** to what.

Orchestration — centralized

A central **saga orchestrator** executes the steps, telling each participant when to run its local transaction (the conductor, or an air-traffic controller).

In practice: the orchestrator calls Order, waits, calls Payment, waits, calls Inventory; if Inventory fails it calls the compensations in reverse. The whole flow is readable in one place.

Choosing between them

	Choreography	Orchestration
Coordination	None — fully decentralized	Central orchestrator
Service autonomy	Higher	Lower — services do as told
Visibility of the flow	Poor — scattered across services	Good — in one place
Error handling	Harder	Easier
Coupling (dependence between parts)	Services coupled to events	Services coupled to the orchestrator
Best for	**Simple** transactions	**Complex** transactions

Rule of thumb: simple transaction → choreography needs less coordination; as complexity grows, **orchestration gives better visibility and control**.

> Choreography's practical failure mode: with six services reacting to events, **nobody can answer "what happens when a customer places an order?"** without reading six **codebases**. The business process exists but is written nowhere — which is why complex flows migrate to orchestration.

Part 4 — Architecture

Local transactions — the backbone

Each runs **within one service and its database**; strung together they maintain consistency across the system. **Every local transaction has an associated compensating transaction.**

Compensating transactions

When a local transaction fails, the saga runs compensations for the previously successful ones, returning the system to a consistent state — retracing your steps to the wrong turn.

The Saga Log

A **saga log** records **every local and compensating transaction executed**, so if the saga is interrupted (a service crashes mid-flow) it can **resume where it left off** or know exactly which compensations to run — a breadcrumb trail back to a consistent state.

Database per service

Each service **owns its database**, so one failure doesn't directly impact others — no **single point of failure**, more robust overall.

Eventual consistency

The ultimate goal: the system **may pass through inconsistent states** but **eventually reaches a consistent one**.

Part 5 — Worked example: online shopping (orchestration)

Scenario: you check out, creating an order; Inventory checks availability; if available it updates stock and the order completes, otherwise the order is cancelled.

The services — each with its own database

```
public class OrderService {
    private OrderDatabase orderDatabase = new OrderDatabase();

    public void createOrder(Order order) {
        orderDatabase.save(order);
    }

    public void cancelOrder(String orderId) {
        // COMPENSATING TRANSACTION for createOrder
    }
}
```

```
public class InventoryService {
    private InventoryDatabase inventoryDatabase = new InventoryDatabase();

    public boolean checkAndReduceInventory(Order order) {
        // local transaction — returns false if stock is insufficient
    }

    public void addBackInventory(Order order) {
        // COMPENSATING TRANSACTION for checkAndReduceInventory
    }
}
```

The pattern in miniature: each service has its **own database**, and each forward operation is **paired with an inverse**:

Forward action	Compensating action
`createOrder`	`cancelOrder`
`checkAndReduceInventory`	`addBackInventory`

That pairing *is* the Saga pattern. Everything else is plumbing to invoke them in the right order.

The orchestrator

```
public class OrderOrchestrator {
    private OrderService orderService = new OrderService();
    private InventoryService inventoryService = new InventoryService();

    public void processOrder(Order order) {
        orderService.createOrder(order);

        if (!inventoryService.checkAndReduceInventory(order)) {
            orderService.cancelOrder(order.getId());   // compensate
        }
    }
}
```

Create the order; try to reduce inventory; if that returns `false`, **undo the order**. Forward and compensation paths are both visible in one method — orchestration's main selling point.

Handling mid-saga failures

What if a service fails partway? That's what the **Saga Log** is for: record each step, and after a failure consult the log and run the needed compensations.

> This example has **no saga log**. A crash between `createOrder` and `checkAndReduceInventory` leaves an order in limbo forever, with no stock reserved and nothing to fix it. Real implementations persist saga state before every step (Part 13D).

What the example is and isn't

A basic illustration — no retries, no **asynchronous messaging**, no log. But the value holds: **split a large transaction into independent parts that can be executed and compensated separately.**

Part 6 — Issues and considerations

Complexity

Each service must now provide **both** its local transaction **and** a compensation. **Concrete difficulty:** if a **discount was applied** and the order is then cancelled for lack of stock, the Order Service's compensation must unwind the discount too — perhaps returning a single-use voucher. Complexity creeps in fast; careful design avoids a tangled web of services.

Longer latency

Each step is a network call, slower than local calls, so sagas have **longer latency than traditional transactions** — what a monolith does in one quick database transaction now crosses the network.

Application consistency — not just data consistency

The most important point in the chapter. Saga restores **data** consistency by compensating — but what about the **business process**?

> If the inventory check fails the order is cancelled — **but cancelling an order is not the same as never having placed it.**

The customer may have a confirmation email; an **audit record** shows the order existed; **analytics** counted it; a warehouse worker saw it on screen. **Compensation is not time travel**: it makes the data consistent but can't un-happen the visible intermediate state. Design the saga with that in mind.

Idempotency

An operation can **fail after executing but before reporting success**, then be retried — so steps must be **idempotent** (repeating has the same effect as once). If `checkAndReduceInventory` reduced stock but failed before reporting, the retry must **not reduce it again** — typically by recording the **saga ID** (unique ID of one saga run) and refusing to apply the same operation twice. *(Same technique as idempotency keys.)*

Increased load

Maintaining a **saga log** means **extra reads and writes in every service**, raising load and possibly response times — the Order Service now handles orders *and* its saga log.

It's a trade-off

Consistency versus complexity, speed versus distributed transactions — understanding the trade-offs is what lets you decide well.

Part 7 — Use cases

E-commerce systems

Order creates the order, **Payment** charges, **Inventory** checks and reduces stock. **If the inventory check fails**, the order is cancelled **and the payment refunded** — a compensating transaction, not a rollback: the charge really happened and appears on the statement; the refund is a second, separate movement of money.

Travel booking systems

Flight, hotel and car rental must **succeed together or fail as a whole**, but they're **independent services, possibly owned by different companies**.

> The clearest case for Saga: there's no shared database, so one ACID transaction across an airline, a hotel chain and a car rental firm is impossible. Compensation (cancel the flight, cancel the hotel) is the *only* mechanism.

Part 8 — System design examples

Online food delivery

Order created → restaurant confirms → delivery partner assigned → payment processed. **What if the restaurant runs out of an item?** Cancel the order and refund the payment; the saga keeps data consistent and handles the failure gracefully.

Internet banking

Transfer from savings to checking: **debit savings**, **credit checking** — if one fails, the other must not stand.

> Without a saga, a failure in the credit step could leave **money disappearing into thin air** — debited and never arriving.

Part 9 — Saga vs. two-phase commit (2PC)

Not in the course, but you'll be asked. **2PC** makes a transaction atomic across databases: a **coordinator** asks every participant "can you commit?" (phase 1) and, if all say yes, tells them "commit now" (phase 2). Background: § Distributed transactions.

	2PC	Saga
Consistency	Strong (genuinely ACID)	Eventual
Locks	Holds locks across all participants until commit	No distributed locks
Availability	**Blocks** if the coordinator fails mid-protocol	Stays available
Rollback	Automatic	Manual — you write the compensating logic
Suits	Short transactions in one trust domain (systems run by one organisation that trust each other)	Long-lived, cross-service, cross-company

Why microservices generally avoid 2PC: it holds locks for the whole distributed operation (Part 1's problem); if the coordinator dies between phases, participants hold locks with no instruction — a

genuinely stuck system; and every participant must support the protocol, which external companies won't.

Part 10 — Common beginner misconceptions

- **"A compensating transaction is a rollback."** A rollback erases history; a compensation is a **new forward action that offsets the previous one**. The refund appears beside the charge; both are real.
- **"Every step can be compensated."** You can't un-send an email, un-ship a package or un-launch a rocket. Put such steps **last**, or design a real-world compensation ("please ignore that email").
- **"Sagas give you ACID across services."** They explicitly don't: you give up isolation entirely — other transactions **can see** your half-finished saga. That's the price of no distributed locks.
- **"Choreography is simpler, so always start there."** For two or three services. Beyond that the invisible business process becomes a serious maintenance problem.
- **"The orchestrator is a single point of failure, so choreography is safer."** The orchestrator can be made highly available with persisted state. Choreography's risk isn't availability — it's comprehensibility.
- **"Eventual consistency means the data might just be wrong."** It means a *window* during which observers may see intermediate state; it converges. The design question is whether the business tolerates that window.

Part 11 — Check your understanding

1. Why can't you use one ACID transaction across three microservices?
2. Why is holding database locks across a distributed transaction impractical?
3. What is the pairing that defines the Saga pattern?
4. Why is "cancelling an order is not the same as never placing it" an important point?
5. Why must saga steps be idempotent?
6. Why does travel booking make the strongest case for Saga?

*(Answers: 1 — each service owns its own database, so there's no single **transaction manager** (the component that coordinates commit and rollback) or shared log to coordinate commit and rollback; 2 — the transaction spans network calls taking seconds, during which locked rows block every other user; 3 — each forward local transaction paired with a compensating transaction that undoes it; 4 — Saga restores data consistency but cannot undo visible side effects like confirmation emails and audit records, so the business process must account for the intermediate state; 5 — a step can succeed but fail before reporting success, then be retried, and must not apply its effect twice; 6 — the participants are separate companies, so a shared database transaction is not merely difficult but impossible.)*

Part 12 — Interview soundbite (30 seconds)

> "Saga handles distributed transactions where ACID can't cross service boundaries. Each service owns its database, so there's no shared transaction manager — and holding locks across network calls for seconds would block everyone anyway. Instead you break the business transaction into local transactions, each paired with a compensating transaction; if step three fails you run the compensations for two and one. You get eventual consistency and give up isolation — others can see your half-finished saga. Two flavours: choreography, where services react to each other's events — fine for simple flows but it scatters the process across codebases so nobody can read it; and orchestration, a central coordinator with much better visibility and error handling for complex flows. A saga log persists progress so you can resume or compensate after a crash, written with a transactional outbox so state and events can't diverge. Classic examples: checkout — order,

> payment, inventory, with a refund as compensation — and travel booking across companies, where a shared transaction is impossible. Hard parts: steps must be idempotent, since a step can succeed but fail to report and get retried; and compensation isn't rollback — the refund is a second real money movement, and a cancelled order isn't a never-placed one, because the confirmation email already went out."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/SagaDemo.java`

```
cd demos && ./gradlew runCh10
```

Compensation driven by the log, in reverse order:

```
void compensate(String sagaId) {
    for (String step : log.stepsNeedingCompensation(sagaId)) {   // newest first
        switch (step) {
            case "reserveStock"  -> inventory.release(sagaId);
            case "chargePayment" -> payments.refund(sagaId);
            case "createOrder"   -> orders.cancelOrder(sagaId);
        }
        log.record(sagaId, step, StepState.COMPENSATED);
    }
}
```

Idempotency using the saga ID as the key:

```
void charge(String sagaId, int amount) {
    if (charges.containsKey(sagaId)) {
        return;                      // duplicate - do NOT charge again
    }
    charges.put(sagaId, amount);
    ledger.add("CHARGE  +" + amount);
}
```

B. What you should see

```
=== 2. Step 3 fails -> compensations run in REVERSE ===
      OrderService: order created
      PaymentService: charged 500
      InventoryService: INSUFFICIENT STOCK (have 3, need 99)
    !!! step failed: out of stock -> compensating
      PaymentService: REFUNDED 500 (compensation)
      OrderService: order CANCELLED (compensation)
  order status: CANCELLED | stock: 3

=== 3. Idempotency: the same step retried ===
      PaymentService: charged 150
      PaymentService: duplicate detected, NOT charging again
      PaymentService: duplicate detected, NOT charging again

=== 4. Compensation is NOT rollback ===
  payment ledger: [CHARGE +300, CHARGE +500, REFUND -500, CHARGE +150]
```

Section 4 is the conceptual point made concrete. In the **ledger** (an append-only list of money movements) the £500 charge and £500 refund **both exist**. A rollback would have erased the charge; compensation adds an **offsetting entry**, and the customer sees both lines.

C. Production — orchestration

Hand-rolled orchestration means reinventing **persistence**, **timeouts** and retries. Real options are **workflow engines** that run multi-step processes reliably: **Temporal**, **Camunda/Zeebe**, **AWS Step Functions** (managed), or **Axon Framework** (a Java framework for event-based systems).

Temporal is the most popular code-first choice: your saga is ordinary code and the framework makes it **durable** — if the process crashes mid-saga, it resumes from the last completed step on another **worker**:

```
public class OrderWorkflowImpl implements OrderWorkflow {

    private final OrderActivities activities = Workflow.newActivityStub(
        OrderActivities.class,
        ActivityOptions.newBuilder()
            .setStartToCloseTimeout(Duration.ofSeconds(10))
            .setRetryOptions(RetryOptions.newBuilder()
                .setMaximumAttempts(3)
                .setBackoffCoefficient(2.0)
                .build())
            .build());

    @Override
    public void processOrder(Order order) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            activities.createOrder(order);
            saga.addCompensation(activities::cancelOrder, order.id());   // register the undo

            activities.chargePayment(order);
            saga.addCompensation(activities::refundPayment, order.id());

            activities.reserveStock(order);                              // may fail
        } catch (ActivityFailure e) {
            saga.compensate();      // runs registered compensations in reverse
            throw e;
        }
    }
}
```

The pattern to copy even without Temporal: register each compensation **immediately after** its step succeeds, so the undo list is always exactly correct with no bookkeeping.

Choreography with Kafka — no orchestrator (Kafka basics — **topics**, consumers, `groupId` — are in Event-Driven 14C):

```
@KafkaListener(topics = "order-created", groupId = "payment-service")
public void onOrderCreated(OrderCreated event) {
    try {
        paymentService.charge(event.orderId(), event.amount());
        kafka.send("payment-completed", new PaymentCompleted(event.orderId()));
    } catch (PaymentFailedException e) {
        kafka.send("payment-failed", new PaymentFailed(event.orderId()));   // triggers compensation
    }
}
```

Every step publishes success or failure, and compensation is just another consumer reacting to a failure event. Nobody is in charge — exactly why complex flows get hard to follow.

D. Implementing the saga log properly

The demo's log is in memory, so a crash loses everything. Real implementations persist it **in the same transaction as the business change**, using the **Transactional Outbox** pattern: write outgoing events into an "outbox" table **in the same local database transaction** as the data change, and publish them afterwards. (In the SQL, `UUID PRIMARY KEY` is a random unique ID as the row's identifying column; `JSONB` is PostgreSQL's binary JSON type.)

```sql
CREATE TABLE saga_state (
    saga_id       UUID PRIMARY KEY,
    current_step  VARCHAR(50)  NOT NULL,
    state         VARCHAR(20)  NOT NULL,   -- STARTED / COMPLETED / COMPENSATING / DONE
    payload       JSONB        NOT NULL,
    updated_at    TIMESTAMP    NOT NULL
);

CREATE TABLE outbox (
    id          UUID PRIMARY KEY,
    aggregate   VARCHAR(50),
    event_type  VARCHAR(100),
    payload     JSONB,
    published   BOOLEAN DEFAULT FALSE
);
```

```java
@Transactional
public void completeStep(String sagaId, String step, Event next) {
    sagaRepo.markCompleted(sagaId, step);      // both writes
    outboxRepo.save(next);                     // in ONE local transaction
}
// A separate poller (or Debezium CDC) publishes unpublished outbox rows to Kafka
```

`@Transactional` runs the method in one database transaction. The relay is either a **poller** (a process checking the table for unpublished rows) or **Debezium CDC** (change data capture: streaming new rows straight from the database's change log — see Strangler Fig 13D).

> **Why the outbox exists.** Writing to the database and then publishing to Kafka as two separate operations means a crash between them leaves them permanently inconsistent — the step completed but nobody was told (or an event announced a change that rolled back). Writing the event into the same database in the same transaction makes the two atomic; a separate process relays it. Relays can redeliver, so consumers must be idempotent. This is the standard solution — know it by name. It applies to **any** service that changes state and publishes an event, not just sagas.

E. Implementation pitfalls

1. Compensations that aren't idempotent. Compensations get retried too; refunding twice is as bad as charging twice. Guard with the saga ID.

2. Compensations that can fail permanently. If the refund API is down: retries with **backoff**, ultimately a **dead-letter queue** (holding area for messages that keep failing) **plus human intervention**. A saga stuck half-compensated is the worst state — alert on it.

3. An uncompensatable step in the middle. You can't un-send an email or un-ship a parcel. **Irreversible actions come last**, after everything that could fail has succeeded.

4. An in-memory saga log. A crash leaves **orphaned** state (half-finished records nothing will clean up). Persist before each step.

5. Writing to the database and publishing separately. Use the outbox — the single most common correctness bug in event-driven sagas.

6. Assuming isolation. Sagas have **no isolation**: a customer may see an order about to be cancelled. Design the **UI** for it ("processing", not "confirmed").

7. Choreography for a complex flow. Six services reacting to each other's events means nobody can say what happens on order placement. Move to orchestration past three or four steps.

8. No timeouts on saga steps. A step that never responds hangs the saga forever. Give every step a deadline, after which it's failed and compensation begins.

Related patterns

- **Event-Driven Architecture** — the messaging substrate choreography sagas run on
- **CQRS** — frequently paired with sagas in **event-sourced** systems (which store every change as an event rather than just the current state)
- **Retry** — idempotency concerns are shared; retries happen within saga steps
- **Circuit Breaker** — protects individual saga participants